Python Code for Problem 2

```python
import numpy as np
from scipy.special import digamma, gammaln
import pandas as pd
import matplotlib.pyplot as plt
import os

def read_files(folder):
    X_set1 = pd.read_csv(os.path.join(folder, 'X_set1.csv'), header=None).values
    X_set2 = pd.read_csv(os.path.join(folder, 'X_set2.csv'), header=None).values
    X_set3 = pd.read_csv(os.path.join(folder, 'X_set3.csv'), header=None).values

    y_set1 = pd.read_csv(os.path.join(folder, 'y_set1.csv'), header=None).values.flatten()
    y_set2 = pd.read_csv(os.path.join(folder, 'y_set2.csv'), header=None).values.flatten()
    y_set3 = pd.read_csv(os.path.join(folder, 'y_set3.csv'), header=None).values.flatten()

    z_set1 = pd.read_csv(os.path.join(folder, 'z_set1.csv'), header=None).values.flatten()
    z_set2 = pd.read_csv(os.path.join(folder, 'z_set2.csv'), header=None).values.flatten()
    z_set3 = pd.read_csv(os.path.join(folder, 'z_set3.csv'), header=None).values.flatten()
    return X_set1, X_set2, X_set3, y_set1, y_set2, y_set3, z_set1, z_set2, z_set3

def run_vi(X, y, a0=1e-16, b0=1e-16, e0=1, f0=1, max_iter=500, tol=1e-6):
    N, d = X.shape

    # 1. Initialize expectations
    E_alpha = np.full(d, a0 / b0)
    E_lambda = e0 / f0

    # Precompute XTX and XTy for speed
    XTX = X.T @ X
    XTy = X.T @ y
    # Precompute constant part of a_k and e_N
    ak = a0 + 0.5
    eN = e0 + N / 2.0

    # Storage for tracking ELBO
    elbo_history = []

    for i in range(max_iter):
        # --- STEP 1: Update q(w) ---
        # Precision matrix (Sigma inverse)
        precision_w = E_lambda * XTX + np.diag(E_alpha)
        Sigma_w = np.linalg.inv(precision_w)
        mu_w = Sigma_w @ (E_lambda * XTy)

        # Expectations needed for other updates
        E_wwT = Sigma_w + np.outer(mu_w, mu_w)
        E_wk2 = np.diag(E_wwT)

        # --- STEP 2: Update q(alpha_k) ---
        # ak is constant, only bk changes
        bk = b0 + 0.5 * E_wk2
        E_alpha = ak / bk
        E_log_alpha = digamma(ak) - np.log(bk)

        # --- STEP 3: Update q(lambda) ---
        # Note: E[(y - Xw)^2] = yTy - 2yX*mu_w + Tr(XTX * E[wwT])
        yTy = np.dot(y, y)
        sum_sq_err = yTy - 2 * np.dot(mu_w, XTy) + np.trace(XTX @ E_wwT)
        # eN is constant, only fN changes
        fN = f0 + 0.5 * sum_sq_err
        E_lambda = eN / fN
        E_log_lambda = digamma(eN) - np.log(fN)

        # --- STEP 4: Calculate exact variational objective function ---
        # part (a): Joint expectations
```

```
        log_p_y = -(N/2)*np.log(2*np.pi) + (N/2)*E_log_lambda - 0.5*E_lambda*sum_sq_err
        log_p_w = -(d/2)*np.log(2*np.pi) + 0.5*np.sum(E_log_alpha) - 0.5*np.sum(E_alpha * E_wk2)
        log_p_alpha = d*(a0*np.log(b0)-gammaln(a0)) + (a0-1)*np.sum((a0 - 1) * E_log_alpha - b0 * E_alpha)
        log_p_lambda = e0*np.log(f0) - gammaln(e0) + (e0-1)*E_log_lambda - f0*E_lambda
        log_joint = log_p_y + log_p_w + log_p_alpha + log_p_lambda

        # part (b): Entropies
        h_w = (d/2)*np.log(2*np.pi + 1) + 0.5*np.log(np.linalg.det(Sigma_w))
        h_alpha = d*(ak + gammaln(ak) + (1-ak)*digamma(ak)) - np.sum(np.log(bk))
        #h_alpha = np.sum(ak - np.log(bk) + gammaln(ak) + (1-ak)*digamma(ak))
        h_lambda = eN - np.log(fN) + gammaln(eN) + (1-eN)*digamma(eN)
        entropies = h_w + h_alpha + h_lambda

        current_elbo = log_joint + entropies
        elbo_history.append(current_elbo)

    return mu_w, Sigma_w, E_alpha, E_lambda, elbo_history


# 1. Run your inference
def plot_variational_objective(elbo_history, dataset_name):
    plt.figure(figsize=(10, 5))
    plt.plot(elbo_history, color='royalblue', linewidth=2, marker='o', markersize=4)
    plt.title(f'Variational Objective Function Convergence for {dataset_name}', fontsize=14)
    plt.xlabel('Iteration', fontsize=12)
    plt.ylabel('ELBO', fontsize=12)
    plt.grid(True, linestyle='--', alpha=0.7)
    plt.show()


def plot_inv_e_alpha(E_alpha, dataset_name):
    inv_E_alpha = 1.0 / np.maximum(E_alpha, 1e-15)
    plt.figure(figsize=(10, 5))
    markerline, stemlines, baseline = plt.stem(range(len(inv_E_alpha)), inv_E_alpha)
    plt.setp(markerline, marker='o', color='royalblue', markersize=6)
    plt.setp(stemlines, color='royalblue', linewidth=1, alpha=0.6)
    plt.setp(baseline, color='black', linewidth=1) # This styles the stem's baseline
    plt.title(f"$1/E[\\alpha_k]$ for {dataset_name}", fontsize=14)
    plt.xlabel("k", fontsize=12)
    plt.ylabel("$1/E[\\alpha_k]$", fontsize=12)
    plt.show()


folder = '/Users/jihyunpark/Documents/Columbia/2026 Spring/Bayesian/HW2/data_csv'
X_set1, X_set2, X_set3, y_set1, y_set2, y_set3, z_set1, z_set2, z_set3 = read_files(folder)

# (a)
mu_w1, Sigma_w1, E_alpha1, E_lambda1, elbo_history1 = run_vi(X_set1, y_set1)
plot_variational_objective(elbo_history1, "Dataset 1")
mu_w2, Sigma_w2, E_alpha2, E_lambda2, elbo_history2 = run_vi(X_set2, y_set2)
plot_variational_objective(elbo_history2, "Dataset 2")
mu_w3, Sigma_w3, E_alpha3, E_lambda3, elbo_history3 = run_vi(X_set3, y_set3)
plot_variational_objective(elbo_history3, "Dataset 3")

#(b)
plot_inv_e_alpha(E_alpha1, "Dataset 1")
plot_inv_e_alpha(E_alpha2, "Dataset 2")
plot_inv_e_alpha(E_alpha3, "Dataset 3")

#(c)
print(f"Dataset 1: {E_lambda1:.3f}")
print(f"Dataset 2: {E_lambda2:.3f}")
print(f"Dataset 3: {E_lambda3:.3f}")


#(d) Predictions and comparison to ground truth
def plot_zi_yi(ax, X, y, z, mu_w, dataset_name):
    # (d) Predictions: sort by z for clean line plot
    y_hat = X @ mu_w
```

```
    sort_idx = np.argsort(z)
    z_s, y_hat_s = z[sort_idx], y_hat[sort_idx]
    sinc_s = 10 * np.sinc(z_s / np.pi)  # ground truth: 10*sin(z)/z

    ax.scatter(z, y, s=10, alpha=0.5, color="gray", label=r"$(z_i, y_i)$")
    ax.plot(z_s, y_hat_s, color="blue", label=r"$\hat{y}_i$")
    ax.plot(z_s, sinc_s, color="red", label=r"$10\,\mathrm{sinc}(z_i)$")
    ax.set_xlabel("z")
    ax.set_ylabel("y")
    ax.set_title("Predictions vs Ground Truth: " + dataset_name)
    ax.legend(fontsize=7)

fig, axs = plt.subplots(3, 1, figsize=(10, 15))
plot_zi_yi(axs[0], X_set1, y_set1, z_set1, mu_w1, "Dataset 1")
plot_zi_yi(axs[1], X_set2, y_set2, z_set2, mu_w2, "Dataset 2")
plot_zi_yi(axs[2], X_set3, y_set3, z_set3, mu_w3, "Dataset 3")
plt.tight_layout()
plt.show()
```